



C1

How each kind of AI breaks

~50 min · four failure modes · the 2x2 · the review budget · the four layers

BY THE END OF THIS CAPSULE YOU WILL BE ABLE TO

Anticipate the characteristic failure of each kind of AI you deploy, say whether it will be visible and whether it can be undone, and put the control that answers THAT failure rather than the one you find easiest to build.

Which kind of AI produces a wrong answer that nothing anywhere flags?

A · A discriminative classifier

B · A large language model

C · An agent with tools

D · Two of the above

Read it carefully — “nothing anywhere flags” is doing the work in this question.

There is no such thing as “AI failure”

PRE-READ 1

Four kinds of AI, four different things we ask of them — and therefore four different errors.

What you ask for decides how it fails. A classifier is asked for a LABEL, and a label is a compressed claim about a population. A language model is asked for TEXT, and text is judged by whether it reads correctly. An agent is asked for an OUTCOME, and it gets there by acting. An embodied system is asked for a MOVEMENT, and a movement changes the physical world.

So: the failure mode is a property of the contract, not of the algorithm.

The control that catches one is silent on the others. This is the practical consequence, and the reason the next twenty minutes exist. A team that has built excellent monitoring is not protected against an irreversible action. A team with excellent gates is not protected against a confident wrong answer.

Takeaway Four contracts, four errors, four controls — and almost every first design builds one of the four.

Discriminative ML — the silent statistical failure

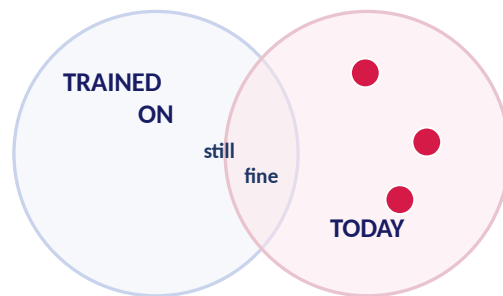
It keeps returning confident labels. The world moved. Nothing says so.

The confidence score is not about the world. It reports the model's fit to its TRAINING distribution, not whether that distribution still describes reality — and accuracy can degrade for months with no error message.

Why it is invisible: the failure is statistical, so no one prediction looks wrong.

And it concentrates. Damage shows only in aggregate, and only if you are aggregating. It rarely spreads evenly: it lands on a subgroup, which the headline number is perfectly capable of hiding.

Next slide: the arithmetic of exactly that.



CONFIDENT • WRONG • SILENT

every dot is a subgroup the model has never seen — a new device, a new dialect, a new claim code. It scores each one anyway, and reports nothing.

Takeaway Conventional software fails loudly. That is the monitoring instinct which does not transfer.

The headline number is an average, and averages hide

WORKED

PRE-READ 1

One model, one month, two ways of reporting the same performance.

What gets reported	The number	What it conceals
Overall accuracy	94%	Looks like a healthy model. This is the number on the dashboard.
Accuracy on the largest segment	96%	Most of the volume — it pulls the average up on its own.
Accuracy on a 6%-of-volume subgroup	71%	Roughly one in three of those customers gets a wrong decision — and the headline never moves.
What it takes to see it	Slicing	Evaluate separately by segment, geography, channel, language, device. It only ever finds what you thought to slice by.

Takeaway A model can be 94% right and 71% right at the same time. Only one of those numbers reaches a meeting.

Large language models — the fluent per-instance failure

NIST calls it CONFABULATION, and the word is better than hallucination.

HALLUCINATION. The popular word, and the wrong picture. It implies a malfunction — a system perceiving something that is not there. Nothing has malfunctioned.

CONFABULATION. NIST AI 600-1's term: the generation of confidently stated erroneous content. Borrowed from neurology, where a patient produces a fluent, sincere, false account with no intent to deceive.

Which is why the second word is the right one: this is the system working. A language model is trained to produce text that is LIKELY. A fluent false statement and a fluent true statement are both likely. Correctness is not a variable the model optimises, so it is not a variable the output reports on — and it does not hedge, slow down or lower its confidence. A confabulated citation arrives in exactly the register of a real one.

For deployment: you cannot detect a confabulation by reading it. The output carries no distress signal at all.

NIST AI 600-1 lists twelve generative-AI risks and confabulation is the second, after CBRN information — the list is alphabetical, not ranked.

Takeaway Fluency and correctness come out of one mechanism. That is the whole problem, in one sentence.

Agents — the failure that has already happened

Week 4 gave you the arithmetic. Here is the other half of it, and what to do about it.

A wrong answer can be thrown away. A wrong action cannot. $0.95^{20} \approx 36\%$ — a 95% per-TURN success rate over twenty turns. But the number is not the point today. Turn two is conditioned on turn one, so an early error is not an error, it is a PREMISE — the agent then reasons competently from being wrong, for another seventeen turns.

And the action is done: the email is sent, the record updated, the refund issued, the file deleted.

Week 4 gave you four moves against exactly this. ① CUT THE TURNS — the biggest win, and why parallel calls exist. ② SHAPE THE TOOL INTERFACE — filter at source, cap results; 246 tokens of stale notes is a failure you designed in. ③ POKA-YOKE THE SIGNATURE — draft_email separate from send_email, dry_run=True by default, enums not strings. ④ CONTROL THE LOOP — dedupe, a step cap, a budget ceiling, and the gate in front of the tool that touches the world.

Week 4, in one line: the model cannot decline to obey a type. A prompt is only a request.

Takeaway Not “the agent was wrong” — “wrong at turn three, and coherent for the rest of the run”.

Embodied AI — the failure with no review step

Every control so far assumed a gap between the decision and its consequence.

The gap is what review lives in. A vehicle, a robot arm, a surgical instrument or a warehouse mover closes that gap to nothing. There is no human in the loop at 60 km/h.

So the burden moves: before deployment — simulation, formal constraint, physical envelope limits, redundant sensing, hard mechanical stops.

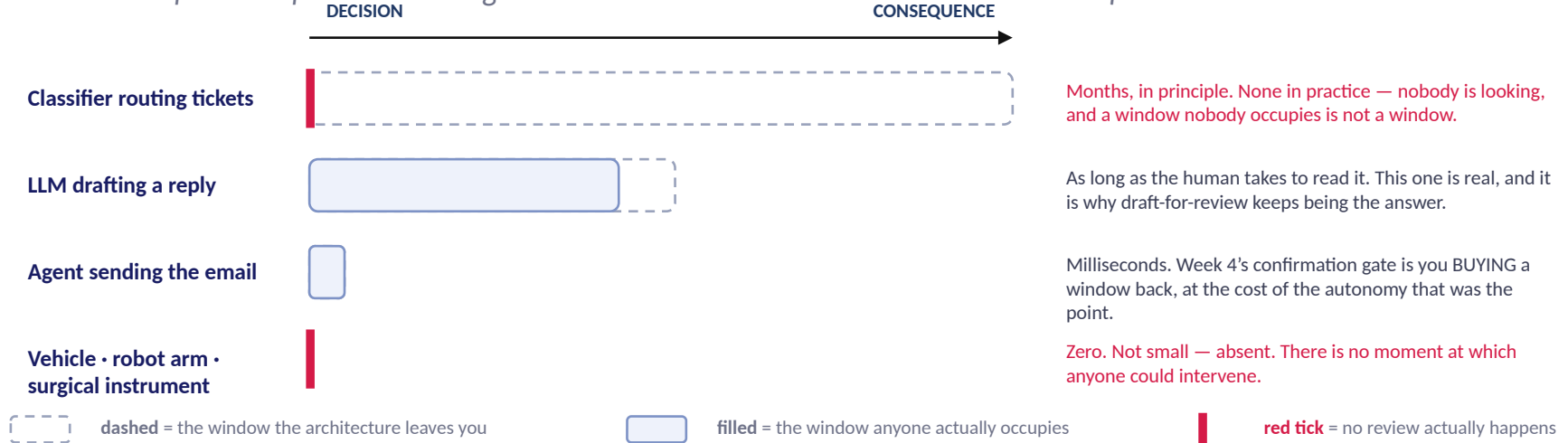
Which is why the literature looks so different. Embodied-AI safety is not a more cautious version of LLM safety. It is a different discipline, because the operational-monitoring layer that carries most of the weight for the other three types can only tell you what went wrong afterwards.

Not more cautious: differently timed.

Takeaway Where the failure is permanent, everything has to happen before it.

The review window has been closing all along

A review step is not a person on an org chart. It is TIME between a decision and its consequence.

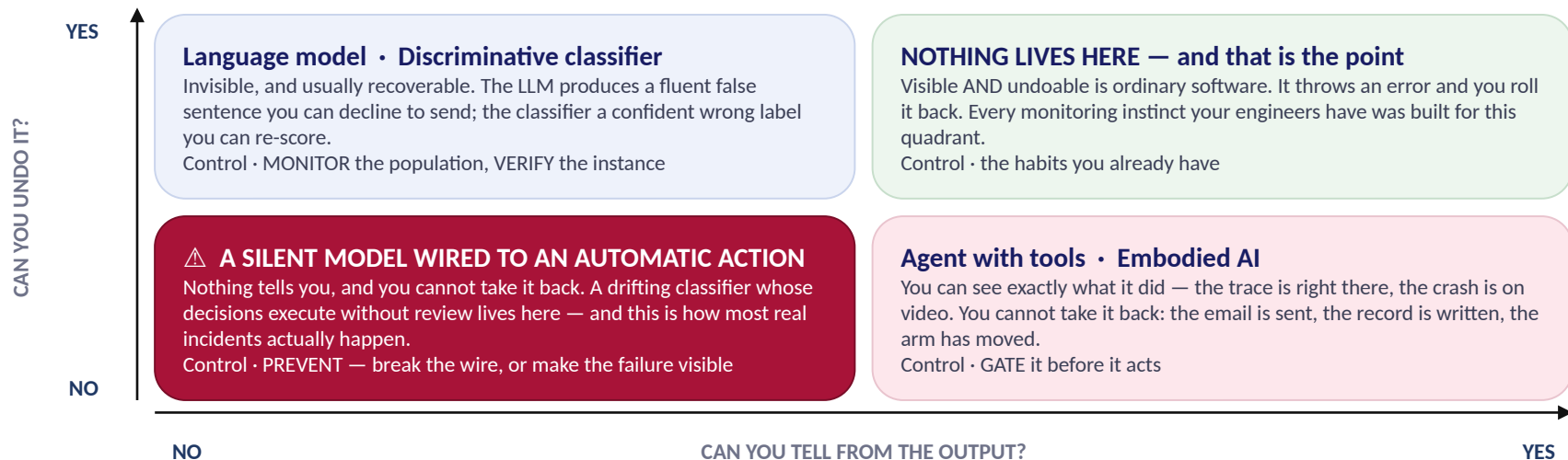


A review step needs three things and time is only the first: the **WINDOW**, the **EVIDENCE** the reviewer sees, and the **AUTHORITY** to stop it. Absent any one, the control does not exist.

Takeaway Embodied AI is not a separate discipline. It is the limiting case of a window closing everywhere.

Two questions sort every failure you will meet

Can you tell from the output alone that it failed — and can you undo it?



Read the grid for COMPONENTS, not for products. One Week-4 build spans two of these: retrieval and generation sit TOP-LEFT; the tool that writes sits BOTTOM-RIGHT.

Takeaway The dangerous quadrant is not the famous one. It is bottom-left: invisible AND irreversible.

The two-question diagnostic — in this order

Run it on every component, not on the product. The order matters.

1 • If this is wrong, will anything tell me?

If NO, you have a monitoring problem. No guardrail at the boundary will help, because the boundary sees a well-formed output of the expected type.

Answers the INVISIBLE failure

2 • If this is wrong, can I undo it?

If NO, you have a gating problem. No amount of monitoring will help, because by the time the monitor fires the thing has already happened.

Answers the PERMANENT failure

Takeaway Monitoring and gating are not substitutes. A team with only one is protected against half its failures.

“A human will check it” — priced

WORKED

PRE-READ 1

50,000 tickets a month • 95% correct • review costs \$1 per output • a shipped error costs \$25

Review policy	Outputs read	Errors shipped	The monthly bill — review + shipped errors
Review every output	50,000	0	$50,000 \times \$1 = \$50,000$ to review • \$0 in errors → \$50,000

ASSUMPTIONS, stated so you can argue with them: review takes 2 minutes at a US\$30/hour loaded rate, so US\$1.00 per output • a wrong answer that reaches the customer costs US\$25 — one repeat contact plus the rework.

Takeaway Start with the arithmetic, not the intention: $50,000 \times 5\% = 2,500$ wrong answers every month.

“A human will check it” — priced

WORKED

PRE-READ 1

50,000 tickets a month • 95% correct • review costs \$1 per output • a shipped error costs \$25

Review policy	Outputs read	Errors shipped	The monthly bill — review + shipped errors
Review every output	50,000	0	$50,000 \times \$1 = \$50,000$ to review • \$0 in errors → \$50,000
Review a random 10%	5,000	2,250	\$5,000 to review • $2,250 \times \$25 = \$56,250$ → \$61,250

ASSUMPTIONS, stated so you can argue with them: review takes 2 minutes at a US\$30/hour loaded rate, so US\$1.00 per output • a wrong answer that reaches the customer costs US\$25 — one repeat contact plus the rework.

Takeaway A tenth of the cost catches a tenth of the errors — and the bill goes UP, not down.

“A human will check it” — priced

WORKED

PRE-READ 1

50,000 tickets a month • 95% correct • review costs \$1 per output • a shipped error costs \$25

Review policy	Outputs read	Errors shipped	The monthly bill — review + shipped errors
Review every output	50,000	0	$50,000 \times \$1 = \$50,000$ to review • \$0 in errors → \$50,000
Review a random 10%	5,000	2,250	\$5,000 to review • $2,250 \times \$25 = \$56,250$ → \$61,250
Review a targeted 5%	2,500	0 ... 2,375	\$2,500 to review • \$0 to \$59,375 in errors → \$2,500 to \$61,875

BREAK-EVEN, so nobody has to defend the \$25: reviewing everything beats sampling 10% as soon as a shipped error costs more than \$20. You may not know your true figure — but you can almost always say whether it is above or below twenty dollars.

Takeaway The signal is worth up to \$59,375 a month. The reviewers are worth \$2,500. Buy the signal.

Three signals that actually exist — and what each costs

Selective review is only possible if you manufacture the signal. None of these is free.

Signal	How it works	What it costs	Where it is blind
Self-consistency	Sample the same question three times at non-zero temperature; flag disagreement.	Triples your token cost on every request.	Silent when the model is CONSISTENTLY wrong — when the error came from training data, not from sampling.
Grounded verification	Require every claim to trace to a retrieved passage, and check the trace.	Needs sources; converts the problem into a retrieval-quality problem.	Only covers claims that HAVE a source. Summarising, drafting and deciding do not.
Structural validation	Demand JSON against a schema; check enums, ranges, that cited IDs exist.	Nearly free — it runs in your own code.	Validates FORM. A well-formed lie passes every time.

Takeaway Do your outputs have sources? If yes, ground them. If no, be honest that you are estimating.

THINK · PAIR · SHARE · 5 MIN

Take a system you own. Put each of its components on the 2×2.

1 · Name three components — retrieval, generation, and anything that WRITES

2 · For each: if it is wrong, will anything tell you?

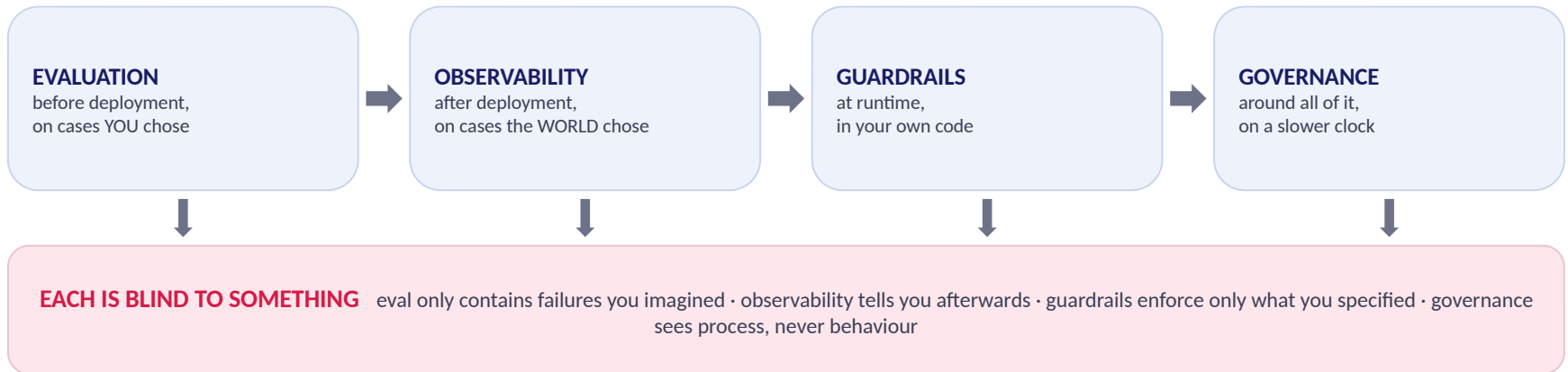
3 · For each: if it is wrong, can you undo it?

4 · Which quadrant currently has no control in it?

2 min alone · 3 min in pairs · we take two · question 4 is the one to bring back

Four operational layers — and what each one cannot see

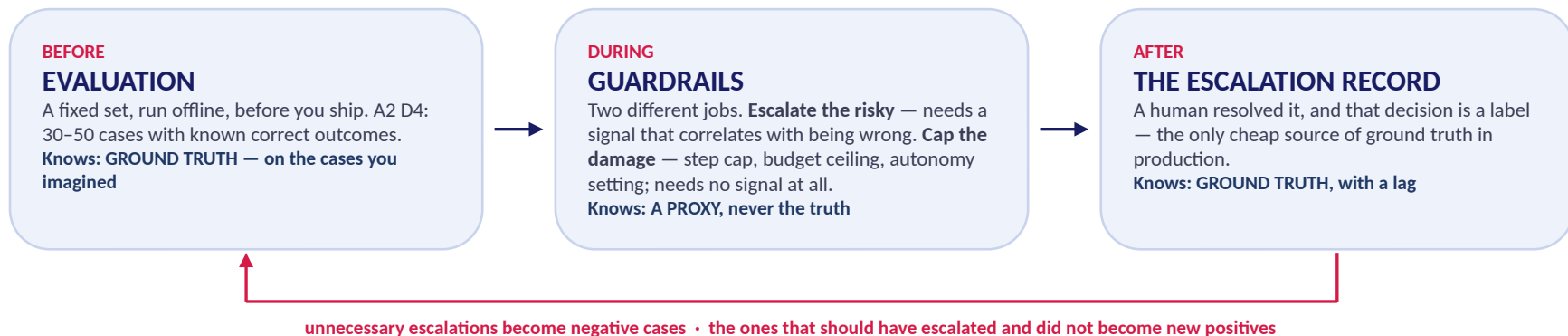
They are not alternatives. Each catches a class of problem the others are blind to.



Takeaway A framework tells you that you did the recognised things. It cannot tell you your system is safe.

Three tenses — and the arrow nobody draws

Evaluation runs before. Guardrails run during. And the escalations become the next evaluation set.



None of the three sees a failure that is silent, unescalated and unmeasured. That is the bottom-left cell again — and the answers there are the same two: break the wire, or sample and label a slice.

Takeaway Three tenses, one loop. Cut the return arrow and your eval set freezes while the traffic moves.

P-U-R-E, closed out

WEEK 2

PRE-READ 1

Four words from Week 2 — a specification, and one of them has no layer at all.

DBS asked	What answers it	What that means in practice
Purposeful	GOVERNANCE	Does this use case have a stated, approved purpose — and a named owner?
Unsurprising	OBSERVABILITY	Is behaviour in production consistent with behaviour in test?
Respectful	EVALUATION, sliced	Have you measured performance on the subgroup, not just the average? (slide 6)
Explainable	⚠ NO LAYER — ARCHITECTURE	No operational layer can add it. Either the decision path is inspectable by construction or it is not — which is why draft-for-review keeps reappearing as the answer.

And GUARDRAILS answer none of the four — because all four ask whether something is TRUE of your system, and a guardrail does not answer questions, it acts.

Takeaway Three of the four are answered by a layer. The fourth was decided before any layer existed.

“Human in the loop” — stated properly

Three things, all three required. Miss one and the control does not exist.

WINDOW

how long between the decision and the consequence

EVIDENCE

what the reviewer actually sees at that moment

AUTHORITY

what they are permitted to do about it

The design	Window	Evidence	Authority
Draft reply, the agent sends it	As long as she takes to read it	The message and the draft, together	She edits it, or discards it
Fraud model auto-blocks the card	NONE — the block already happened	Full transaction detail	They can unblock it
Analyst approving 400 decisions a shift	Nothing executes until approved	NONE — a score and an Approve button	He can reject any of them
Ops can see the model drifting	Weeks of it	They built the dashboard	NONE — pausing it needs a VP

All three can be present and the control still decays — AUTOMATION BIAS: scrutiny declines after a run of correct outputs, and Singapore’s agentic framework names it explicitly.

Takeaway Every failing row has two of the three. Two out of three is not a control, it is a record.

FOR BUILDERS • what to aggregate, and what to slice by

FOR BUILDERS

PRE-READ 1

The monitoring that catches a silent classifier failure — concretely.

Watch	Why	It fires when
Input distribution	DATA DRIFT is detectable WITHOUT labels — you need no ground truth to see that the inputs changed.	Feature distributions move against a reference window.
Output class balance	A change in the predicted mix is a real signal about a model nobody has told anything.	The mix shifts with no business explanation.
Override and re-open rate	THE CHEAPEST CONCEPT-DRIFT SIGNAL THERE IS — humans correcting the model, already logged, and it moves months before labels arrive.	Overrides rise with no policy or process change.
Outcome labels, joined back on a lag	The only DIRECT measure of concept drift: join prediction → what actually happened, and state the lag honestly.	Rolling accuracy on arriving labels falls.
Sliced accuracy, on those labels	The average hides the subgroup (slide 6). Slice by segment, geography, channel, language, device.	Any slice degrades while the headline holds.
A frozen held-out set, on a schedule	The cheapest regression test you will run — and the gate on any retrain.	The score drops against a fixed baseline.

Takeaway Data drift needs no labels. Concept drift needs outcomes — and the override rate is one you already have.

Your six weeks, from the attacker's side

OWASP 2026

PRE-READ 2

OWASP Top 10 for LLM Applications, 2026 edition. Four entries, four things you have already met.

OWASP 2026 — what it is	Where you met it	The control that answers it
LLM01:2026 Prompt Injection — someone else's text becomes your model's instructions.	WEEK 3 — you wrote system prompts and watched a model follow them.	None that is clean. C2-B after the break: narrow what it can reach, do not pretend to filter.
LLM03:2026 Excessive Agency — the system can do more than the task requires.	WEEK 4 — five tools, four reads and one write. Ask what the write could do.	WEEK 4 — the draft/send split, dry_run by default, and the gate in front of the write.
LLM06:2026 Unbounded Consumption — runaway cost; the bill is the attack.	WEEKS 3, 4 & 5 — token arithmetic as an attack, and the loop that would not stop.	WEEK 4 — a STEP CAP and a BUDGET CEILING that HALT, not alert, plus dedupe and a timeout. Asking the model to be frugal is not a control.
LLM09:2026 Vector & Embedding Weaknesses — the retrieval layer is attackable and can leak across tenants.	WEEK 2 — a corpus you wrote yourself. The WRITE path was never in scope.	Partition per tenant; treat anything a tool or a third party put in the store as untrusted text — which is row 1 again.

None of that write path was Week 2. It bites where ONE INDEX SERVES MANY TENANTS, where connectors ingest Slack, Drive and the open web, and where untrusted agents write into a shared store — Guest Lecture 1 made deep integration the moat, Guest Lecture 2 said permission not prompts, and this is the bill for the first without the second.

Takeaway You already wrote the control for two of these. The other two are the reason for the next two hours.

CAPSULE 1 • TAKEAWAY

Monitor the invisible. Gate the irreversible.

Four kinds of AI, four failures, four controls — and the two questions that tell you which one you need. After the break: what you are obliged to do about it, and then breaking your own system on purpose.